AI travel web app

Generated date: 2026-04-26
Team size: 4
Skill level: Beginner
Timeline: 10 days
Difficulty score: 6/10
Completion confidence: 80%

Project Flow (ASCII)

Project Idea
  |
Planning -> Development -> Testing -> Polish & Demo
  |
Final Deliverables
  |
Milestones Achieved

Executive Summary

This roadmap outlines a Web Dev project plan for AI travel web app with an estimated difficulty of 6/10 and completion confidence of 80%. The plan is structured for steady delivery, incremental validation, and final demo readiness.

Tools & Tech Stack

- React
- Node.js
- Socket.io
- MongoDB
- HTML
- CSS
- JavaScript
- Bootstrap
- Firebase

Day-wise Plan

DAY 1 - Planning

Task 1: Define scope and acceptance checks for Day 1

What: Map the day goals for AI travel web app into 4-5 concrete acceptance checks.

How: Create notes/day_1_scope.md and list measurable checks aligned with the current phase.

Output: A clear day scope document with acceptance checklist ready for execution.

Time: 30 mins

Task 2: Implement the primary planning component

What: Build the highest-priority feature for Day 1 tied to the project milestone.

How: Code the component using the selected stack, run a targeted smoke test, and capture logs/screenshots.

Output: Working feature implementation with test evidence and reproducible run steps.

Time: 90 mins

Task 3: Document progress and blockers for Day 1

What: Summarize completed work, pending items, and blockers found during execution.

How: Update notes/day_1_report.md with bullets, error traces, and next-day handoff notes.

Output: A review-ready daily report that can be used in demo/viva discussion.

Time: 25 mins

Task 4: Implement core planning code path in `src/day_1_feature.py`

What: Write or extend the main logic for AI travel web app in a dedicated module for Day 1.

How: Create `src/day_1_feature.py`, add `build_day_1_feature()` using React, and run local test command.

Output: Module compiles and produces expected output for one realistic AI travel web app scenario.

Time: 1h 15 mins

Task 5: Test critical flow with CLI/API checks for Day 1

What: Verify the key feature path with positive and negative test cases.

How: Run `python -m pytest` (or equivalent) and capture response/trace details for failed and passing cases.

Output: Test evidence file with pass/fail status and corrected issues.

Time: 50 mins

Task 6: Document execution notes in `notes/day_1_execution.md`

What: Record exact implementation steps, commands, and unresolved edge cases.

How: Write notes including commands, function names, and outcomes using Node.js workflow context.

Output: Review-ready day log that another teammate can reproduce end-to-end.

Time: 25 mins

Task 7: Implement core planning code path in `src/day_1_feature.py` (variation 2)

What: Write or extend the main logic for AI travel web app in a dedicated module for Day 1.

How: Create `src/day_1_feature.py`, add `build_day_1_feature()` using React, and run local test command.

Output: Module compiles and produces expected output for one realistic AI travel web app scenario.

Time: 1h 5 mins

Day duration estimate: 6 hours

Key output: Planning deliverable checkpoint 1

DAY 2 - Planning

Task 1: Define scope and acceptance checks for Day 2

What: Map the day goals for AI travel web app into 4-5 concrete acceptance checks.

How: Create notes/day_2_scope.md and list measurable checks aligned with the current phase.

Output: A clear day scope document with acceptance checklist ready for execution.

Time: 30 mins

Task 2: Implement the primary planning component (variation 2)

What: Build the highest-priority feature for Day 2 tied to the project milestone.

How: Code the component using the selected stack, run a targeted smoke test, and capture logs/screenshots.

Output: Working feature implementation with test evidence and reproducible run steps.

Time: 90 mins

Task 3: Document progress and blockers for Day 2

What: Summarize completed work, pending items, and blockers found during execution.

How: Update notes/day_2_report.md with bullets, error traces, and next-day handoff notes.

Output: A review-ready daily report that can be used in demo/viva discussion.

Time: 25 mins

Task 4: Implement core planning code path in `src/day_2_feature.py`

What: Write or extend the main logic for AI travel web app in a dedicated module for Day 2.

How: Create `src/day_2_feature.py`, add `build_day_2_feature()` using React, and run local test command.

Output: Module compiles and produces expected output for one realistic AI travel web app scenario.

Time: 1h 15 mins

Task 5: Test critical flow with CLI/API checks for Day 2

What: Verify the key feature path with positive and negative test cases.

How: Run `python -m pytest` (or equivalent) and capture response/trace details for failed and passing cases.

Output: Test evidence file with pass/fail status and corrected issues.

Time: 50 mins

Task 6: Document execution notes in `notes/day_2_execution.md`

What: Record exact implementation steps, commands, and unresolved edge cases.

How: Write notes including commands, function names, and outcomes using Node.js workflow context.

Output: Review-ready day log that another teammate can reproduce end-to-end.

Time: 25 mins

Task 7: Implement core planning code path in `src/day_2_feature.py` (variation 2)

What: Write or extend the main logic for AI travel web app in a dedicated module for Day 2.

How: Create `src/day_2_feature.py`, add `build_day_2_feature()` using React, and run local test command.

Output: Module compiles and produces expected output for one realistic AI travel web app scenario.

Time: 1h 5 mins

Day duration estimate: 6 hours

Key output: Planning deliverable checkpoint 2

DAY 3 - Development

Task 1: Define scope and acceptance checks for Day 3

What: Map the day goals for AI travel web app into 4-5 concrete acceptance checks.

How: Create notes/day_3_scope.md and list measurable checks aligned with the current phase.

Output: A clear day scope document with acceptance checklist ready for execution.

Time: 30 mins

Task 2: Implement the primary development component

What: Build the highest-priority feature for Day 3 tied to the project milestone.

How: Code the component using the selected stack, run a targeted smoke test, and capture logs/screenshots.

Output: Working feature implementation with test evidence and reproducible run steps.

Time: 90 mins

Task 3: Document progress and blockers for Day 3

What: Summarize completed work, pending items, and blockers found during execution.

How: Update notes/day_3_report.md with bullets, error traces, and next-day handoff notes.

Output: A review-ready daily report that can be used in demo/viva discussion.

Time: 25 mins

Task 4: Implement core development code path in `src/day_3_feature.py`

What: Write or extend the main logic for AI travel web app in a dedicated module for Day 3.

How: Create `src/day_3_feature.py`, add `build_day_3_feature()` using React, and run local test command.

Output: Module compiles and produces expected output for one realistic AI travel web app scenario.

Time: 1h 15 mins

Task 5: Test critical flow with CLI/API checks for Day 3

What: Verify the key feature path with positive and negative test cases.

How: Run `python -m pytest` (or equivalent) and capture response/trace details for failed and passing cases.

Output: Test evidence file with pass/fail status and corrected issues.

Time: 50 mins

Task 6: Document execution notes in `notes/day_3_execution.md`

What: Record exact implementation steps, commands, and unresolved edge cases.

How: Write notes including commands, function names, and outcomes using Node.js workflow context.

Output: Review-ready day log that another teammate can reproduce end-to-end.

Time: 25 mins

Task 7: Implement core development code path in `src/day_3_feature.py` (variation 2)

What: Write or extend the main logic for AI travel web app in a dedicated module for Day 3.

How: Create `src/day_3_feature.py`, add `build_day_3_feature()` using React, and run local test command.

Output: Module compiles and produces expected output for one realistic AI travel web app scenario.

Time: 1h 5 mins

Day duration estimate: 6 hours

Key output: Development deliverable checkpoint 3

DAY 4 - Development

Task 1: Define scope and acceptance checks for Day 4

What: Map the day goals for AI travel web app into 4-5 concrete acceptance checks.

How: Create notes/day_4_scope.md and list measurable checks aligned with the current phase.

Output: A clear day scope document with acceptance checklist ready for execution.

Time: 30 mins

Task 2: Implement the primary development component (variation 2)

What: Build the highest-priority feature for Day 4 tied to the project milestone.

How: Code the component using the selected stack, run a targeted smoke test, and capture logs/screenshots.

Output: Working feature implementation with test evidence and reproducible run steps.

Time: 90 mins

Task 3: Document progress and blockers for Day 4

What: Summarize completed work, pending items, and blockers found during execution.

How: Update notes/day_4_report.md with bullets, error traces, and next-day handoff notes.

Output: A review-ready daily report that can be used in demo/viva discussion.

Time: 25 mins

Task 4: Implement core development code path in `src/day_4_feature.py`

What: Write or extend the main logic for AI travel web app in a dedicated module for Day 4.

How: Create `src/day_4_feature.py`, add `build_day_4_feature()` using React, and run local test command.

Output: Module compiles and produces expected output for one realistic AI travel web app scenario.

Time: 1h 15 mins

Task 5: Test critical flow with CLI/API checks for Day 4

What: Verify the key feature path with positive and negative test cases.

How: Run `python -m pytest` (or equivalent) and capture response/trace details for failed and passing cases.

Output: Test evidence file with pass/fail status and corrected issues.

Time: 50 mins

Task 6: Document execution notes in `notes/day_4_execution.md`

What: Record exact implementation steps, commands, and unresolved edge cases.

How: Write notes including commands, function names, and outcomes using Node.js workflow context.

Output: Review-ready day log that another teammate can reproduce end-to-end.

Time: 25 mins

Task 7: Implement core development code path in `src/day_4_feature.py` (variation 2)

What: Write or extend the main logic for AI travel web app in a dedicated module for Day 4.

How: Create `src/day_4_feature.py`, add `build_day_4_feature()` using React, and run local test command.

Output: Module compiles and produces expected output for one realistic AI travel web app scenario.

Time: 1h 5 mins

Day duration estimate: 6 hours

Key output: Development deliverable checkpoint 4

DAY 5 - Development

Task 1: Define scope and acceptance checks for Day 5

What: Map the day goals for AI travel web app into 4-5 concrete acceptance checks.

How: Create notes/day_5_scope.md and list measurable checks aligned with the current phase.

Output: A clear day scope document with acceptance checklist ready for execution.

Time: 30 mins

Task 2: Implement the primary development component (variation 3)

What: Build the highest-priority feature for Day 5 tied to the project milestone.

How: Code the component using the selected stack, run a targeted smoke test, and capture logs/screenshots.

Output: Working feature implementation with test evidence and reproducible run steps.

Time: 90 mins

Task 3: Document progress and blockers for Day 5

What: Summarize completed work, pending items, and blockers found during execution.

How: Update notes/day_5_report.md with bullets, error traces, and next-day handoff notes.

Output: A review-ready daily report that can be used in demo/viva discussion.

Time: 25 mins

Task 4: Implement core development code path in `src/day_5_feature.py`

What: Write or extend the main logic for AI travel web app in a dedicated module for Day 5.

How: Create `src/day_5_feature.py`, add `build_day_5_feature()` using React, and run local test command.

Output: Module compiles and produces expected output for one realistic AI travel web app scenario.

Time: 1h 15 mins

Task 5: Test critical flow with CLI/API checks for Day 5

What: Verify the key feature path with positive and negative test cases.

How: Run `python -m pytest` (or equivalent) and capture response/trace details for failed and passing cases.

Output: Test evidence file with pass/fail status and corrected issues.

Time: 50 mins

Task 6: Document execution notes in `notes/day_5_execution.md`

What: Record exact implementation steps, commands, and unresolved edge cases.

How: Write notes including commands, function names, and outcomes using Node.js workflow context.

Output: Review-ready day log that another teammate can reproduce end-to-end.

Time: 25 mins

Task 7: Implement core development code path in `src/day_5_feature.py` (variation 2)

What: Write or extend the main logic for AI travel web app in a dedicated module for Day 5.

How: Create `src/day_5_feature.py`, add `build_day_5_feature()` using React, and run local test command.

Output: Module compiles and produces expected output for one realistic AI travel web app scenario.

Time: 1h 5 mins

Day duration estimate: 6 hours

Key output: Development deliverable checkpoint 5

DAY 6 - Development

Task 1: Define scope and acceptance checks for Day 6

What: Map the day goals for AI travel web app into 4-5 concrete acceptance checks.

How: Create notes/day_6_scope.md and list measurable checks aligned with the current phase.

Output: A clear day scope document with acceptance checklist ready for execution.

Time: 30 mins

Task 2: Implement the primary development component (variation 4)

What: Build the highest-priority feature for Day 6 tied to the project milestone.

How: Code the component using the selected stack, run a targeted smoke test, and capture logs/screenshots.

Output: Working feature implementation with test evidence and reproducible run steps.

Time: 90 mins

Task 3: Document progress and blockers for Day 6

What: Summarize completed work, pending items, and blockers found during execution.

How: Update notes/day_6_report.md with bullets, error traces, and next-day handoff notes.

Output: A review-ready daily report that can be used in demo/viva discussion.

Time: 25 mins

Task 4: Implement core development code path in `src/day_6_feature.py`

What: Write or extend the main logic for AI travel web app in a dedicated module for Day 6.

How: Create `src/day_6_feature.py`, add `build_day_6_feature()` using React, and run local test command.

Output: Module compiles and produces expected output for one realistic AI travel web app scenario.

Time: 1h 15 mins

Task 5: Test critical flow with CLI/API checks for Day 6

What: Verify the key feature path with positive and negative test cases.

How: Run `python -m pytest` (or equivalent) and capture response/trace details for failed and passing cases.

Output: Test evidence file with pass/fail status and corrected issues.

Time: 50 mins

Task 6: Document execution notes in `notes/day_6_execution.md`

What: Record exact implementation steps, commands, and unresolved edge cases.

How: Write notes including commands, function names, and outcomes using Node.js workflow context.

Output: Review-ready day log that another teammate can reproduce end-to-end.

Time: 25 mins

Task 7: Implement core development code path in `src/day_6_feature.py` (variation 2)

What: Write or extend the main logic for AI travel web app in a dedicated module for Day 6.

How: Create `src/day_6_feature.py`, add `build_day_6_feature()` using React, and run local test command.

Output: Module compiles and produces expected output for one realistic AI travel web app scenario.

Time: 1h 5 mins

Day duration estimate: 6 hours

Key output: Development deliverable checkpoint 6

DAY 7 - Testing

Task 1: Define scope and acceptance checks for Day 7

What: Map the day goals for AI travel web app into 4-5 concrete acceptance checks.

How: Create notes/day_7_scope.md and list measurable checks aligned with the current phase.

Output: A clear day scope document with acceptance checklist ready for execution.

Time: 30 mins

Task 2: Implement the primary testing component

What: Build the highest-priority feature for Day 7 tied to the project milestone.

How: Code the component using the selected stack, run a targeted smoke test, and capture logs/screenshots.

Output: Working feature implementation with test evidence and reproducible run steps.

Time: 90 mins

Task 3: Document progress and blockers for Day 7

What: Summarize completed work, pending items, and blockers found during execution.

How: Update notes/day_7_report.md with bullets, error traces, and next-day handoff notes.

Output: A review-ready daily report that can be used in demo/viva discussion.

Time: 25 mins

Task 4: Implement core testing code path in `src/day_7_feature.py`

What: Write or extend the main logic for AI travel web app in a dedicated module for Day 7.

How: Create `src/day_7_feature.py`, add `build_day_7_feature()` using React, and run local test command.

Output: Module compiles and produces expected output for one realistic AI travel web app scenario.

Time: 1h 15 mins

Task 5: Test critical flow with CLI/API checks for Day 7

What: Verify the key feature path with positive and negative test cases.

How: Run `python -m pytest` (or equivalent) and capture response/trace details for failed and passing cases.

Output: Test evidence file with pass/fail status and corrected issues.

Time: 50 mins

Task 6: Document execution notes in `notes/day_7_execution.md`

What: Record exact implementation steps, commands, and unresolved edge cases.

How: Write notes including commands, function names, and outcomes using Node.js workflow context.

Output: Review-ready day log that another teammate can reproduce end-to-end.

Time: 25 mins

Task 7: Implement core testing code path in `src/day_7_feature.py` (variation 2)

What: Write or extend the main logic for AI travel web app in a dedicated module for Day 7.

How: Create `src/day_7_feature.py`, add `build_day_7_feature()` using React, and run local test command.

Output: Module compiles and produces expected output for one realistic AI travel web app scenario.

Time: 1h 5 mins

Day duration estimate: 6 hours

Key output: Testing deliverable checkpoint 7

DAY 8 - Testing

Task 1: Define scope and acceptance checks for Day 8

What: Map the day goals for AI travel web app into 4-5 concrete acceptance checks.

How: Create notes/day_8_scope.md and list measurable checks aligned with the current phase.

Output: A clear day scope document with acceptance checklist ready for execution.

Time: 30 mins

Task 2: Implement the primary testing component (variation 2)

What: Build the highest-priority feature for Day 8 tied to the project milestone.

How: Code the component using the selected stack, run a targeted smoke test, and capture logs/screenshots.

Output: Working feature implementation with test evidence and reproducible run steps.

Time: 90 mins

Task 3: Document progress and blockers for Day 8

What: Summarize completed work, pending items, and blockers found during execution.

How: Update notes/day_8_report.md with bullets, error traces, and next-day handoff notes.

Output: A review-ready daily report that can be used in demo/viva discussion.

Time: 25 mins

Task 4: Implement core testing code path in `src/day_8_feature.py`

What: Write or extend the main logic for AI travel web app in a dedicated module for Day 8.

How: Create `src/day_8_feature.py`, add `build_day_8_feature()` using React, and run local test command.

Output: Module compiles and produces expected output for one realistic AI travel web app scenario.

Time: 1h 15 mins

Task 5: Test critical flow with CLI/API checks for Day 8

What: Verify the key feature path with positive and negative test cases.

How: Run `python -m pytest` (or equivalent) and capture response/trace details for failed and passing cases.

Output: Test evidence file with pass/fail status and corrected issues.

Time: 50 mins

Task 6: Document execution notes in `notes/day_8_execution.md`

What: Record exact implementation steps, commands, and unresolved edge cases.

How: Write notes including commands, function names, and outcomes using Node.js workflow context.

Output: Review-ready day log that another teammate can reproduce end-to-end.

Time: 25 mins

Task 7: Implement core testing code path in `src/day_8_feature.py` (variation 2)

What: Write or extend the main logic for AI travel web app in a dedicated module for Day 8.

How: Create `src/day_8_feature.py`, add `build_day_8_feature()` using React, and run local test command.

Output: Module compiles and produces expected output for one realistic AI travel web app scenario.

Time: 1h 5 mins

Day duration estimate: 6 hours

Key output: Testing deliverable checkpoint 8

DAY 9 - Polish & Demo

Task 1: Define scope and acceptance checks for Day 9

What: Map the day goals for AI travel web app into 4-5 concrete acceptance checks.

How: Create notes/day_9_scope.md and list measurable checks aligned with the current phase.

Output: A clear day scope document with acceptance checklist ready for execution.

Time: 30 mins

Task 2: Implement the primary polish & demo component

What: Build the highest-priority feature for Day 9 tied to the project milestone.

How: Code the component using the selected stack, run a targeted smoke test, and capture logs/screenshots.

Output: Working feature implementation with test evidence and reproducible run steps.

Time: 90 mins

Task 3: Document progress and blockers for Day 9

What: Summarize completed work, pending items, and blockers found during execution.

How: Update notes/day_9_report.md with bullets, error traces, and next-day handoff notes.

Output: A review-ready daily report that can be used in demo/viva discussion.

Time: 25 mins

Task 4: Implement core polish & demo code path in `src/day_9_feature.py`

What: Write or extend the main logic for AI travel web app in a dedicated module for Day 9.

How: Create `src/day_9_feature.py`, add `build_day_9_feature()` using React, and run local test command.

Output: Module compiles and produces expected output for one realistic AI travel web app scenario.

Time: 1h 15 mins

Task 5: Test critical flow with CLI/API checks for Day 9

What: Verify the key feature path with positive and negative test cases.

How: Run `python -m pytest` (or equivalent) and capture response/trace details for failed and passing cases.

Output: Test evidence file with pass/fail status and corrected issues.

Time: 50 mins

Task 6: Document execution notes in `notes/day_9_execution.md`

What: Record exact implementation steps, commands, and unresolved edge cases.

How: Write notes including commands, function names, and outcomes using Node.js workflow context.

Output: Review-ready day log that another teammate can reproduce end-to-end.

Time: 25 mins

Task 7: Implement core polish & demo code path in `src/day_9_feature.py` (variation 2)

What: Write or extend the main logic for AI travel web app in a dedicated module for Day 9.

How: Create `src/day_9_feature.py`, add `build_day_9_feature()` using React, and run local test command.

Output: Module compiles and produces expected output for one realistic AI travel web app scenario.

Time: 1h 5 mins

Day duration estimate: 6 hours

Key output: Polish & Demo deliverable checkpoint 9

DAY 10 - Polish & Demo

Task 1: Define scope and acceptance checks for Day 10

What: Map the day goals for AI travel web app into 4-5 concrete acceptance checks.

How: Create notes/day_10_scope.md and list measurable checks aligned with the current phase.

Output: A clear day scope document with acceptance checklist ready for execution.

Time: 30 mins

Task 2: Implement the primary polish & demo component (variation 2)

What: Build the highest-priority feature for Day 10 tied to the project milestone.

How: Code the component using the selected stack, run a targeted smoke test, and capture logs/screenshots.

Output: Working feature implementation with test evidence and reproducible run steps.

Time: 90 mins

Task 3: Document progress and blockers for Day 10

What: Summarize completed work, pending items, and blockers found during execution.

How: Update notes/day_10_report.md with bullets, error traces, and next-day handoff notes.

Output: A review-ready daily report that can be used in demo/viva discussion.

Time: 25 mins

Task 4: Implement core polish & demo code path in `src/day_10_feature.py`

What: Write or extend the main logic for AI travel web app in a dedicated module for Day 10.

How: Create `src/day_10_feature.py`, add `build_day_10_feature()` using React, and run local test command.

Output: Module compiles and produces expected output for one realistic AI travel web app scenario.

Time: 1h 15 mins

Task 5: Test critical flow with CLI/API checks for Day 10

What: Verify the key feature path with positive and negative test cases.

How: Run `python -m pytest` (or equivalent) and capture response/trace details for failed and passing cases.

Output: Test evidence file with pass/fail status and corrected issues.

Time: 50 mins

Task 6: Document execution notes in `notes/day_10_execution.md`

What: Record exact implementation steps, commands, and unresolved edge cases.

How: Write notes including commands, function names, and outcomes using Node.js workflow context.

Output: Review-ready day log that another teammate can reproduce end-to-end.

Time: 25 mins

Task 7: Implement core polish & demo code path in `src/day_10_feature.py` (variation 2)

What: Write or extend the main logic for AI travel web app in a dedicated module for Day 10.

How: Create `src/day_10_feature.py`, add `build_day_10_feature()` using React, and run local test command.

Output: Module compiles and produces expected output for one realistic AI travel web app scenario.

Time: 1h 5 mins

Day duration estimate: 6 hours

Key output: Polish & Demo deliverable checkpoint 10

MVP Features

- User auth
- Realtime messaging

Optional Features

- Typing indicators
- Media sharing
- Read receipts

Risk Register

Risk	Severity	Mitigation
Authentication bypass vulnerability	High	Immediate mitigation owner, add fallback path, a
Broken role-based access control	High	Immediate mitigation owner, add fallback path, a
SQL/NoSQL injection vulnerability	High	Immediate mitigation owner, add fallback path, a
Cross-site scripting exposure	High	Immediate mitigation owner, add fallback path, a
Payment gateway integration failures	High	Immediate mitigation owner, add fallback path, a
Caching stale critical data	Medium	Track in sprint board and review in daily standu
Session expiration bugs	Medium	Track in sprint board and review in daily standu
Unoptimized frontend causing slow loads	Medium	Track in sprint board and review in daily standu
Cross-browser rendering issues	Low	Monitor in review checklist and verify before de
File upload validation bypass	High	Immediate mitigation owner, add fallback path, a
Production misconfiguration	High	Immediate mitigation owner, add fallback path, a
Message loss	Medium	Track in sprint board and review in daily standu

Milestones Timeline

1. Rooms working
2. Realtime stable
3. Notifications added
4. Deploy done

Viva Preparation

Focus on explaining architecture decisions, trade-offs, and risk handling for AI travel web app.

Q: How do websockets work?

A: Prepare a concise 3-4 line technical explanation with one example.

Q: How is message ordering handled?

A: Prepare a concise 3-4 line technical explanation with one example.

Report Summary

This roadmap outlines a Web Dev project plan for AI travel web app with an estimated difficulty of 6/10 and completion confidence of 80%. The plan is structured for steady delivery, incremental validation, and final demo readiness.